

CHƯƠNG 5: THỊ GIÁC MÁY TÍNH

Giới thiệu

Nông nghiệp đã đóng một vai trò quan trọng trong nền kinh tế toàn cầu trong những năm gần đây. Sự gia tăng hơn nữa của dân số đang dẫn đến việc giảm dần diện tích đất nông nghiệp và gây thêm căng thẳng ngày càng tăng cho hệ thống trang trại. Nhu cầu về các phương pháp sản xuất thực phẩm nông nghiệp thành công và lành mạnh đang tăng lên. Các công nghệ cảm biến và động lực sáng tạo cùng các hệ thống thông tin và truyền thông tiên tiến và Trí tuệ nhân tạo phải bổ sung cho các phương pháp quản lý nông nghiệp truyền thống để tăng tốc độ tăng trưởng năng suất trang trại một cách chính xác hơn, qua đó thúc đẩy sản xuất nông nghiệp chất lượng cao và hiệu suất cao. Hệ thống giám sát bằng thị giác máy tính đã trở thành công cụ có giá trị trong hoạt động trang trại trong nhiều thập kỷ qua, và việc sử dụng chúng đã tăng lên đáng kể.

Thị giác máy tính là gì?

Thị giác máy tính có thể được mô tả như một lĩnh vực của AI dùng để trích xuất thông tin từ hình ảnh kỹ thuật số. Loại dữ liệu thu được từ một hình ảnh có thể khác nhau, từ nhận dạng, đo lường không gian điều hướng hoặc các ứng dụng thực tế tăng cường.

Các ứng dụng của thị giác máy tính cũng có thể được mô tả. Thị giác máy tính xây dựng các thuật toán nhận dạng và sử dụng chất lượng của hình ảnh cho các mục đích khác.

Con người nhìn và trải nghiệm thế giới xung quanh một cách vật lý thông qua mắt và não bộ. Thị giác máy tính là khoa học muốn cung cấp cho máy móc hoặc thiết bị một hiệu suất tương tự, nếu không muốn nói là tốt hơn.

Mục đích của thị giác máy tính là tự động thu thập, diễn giải và hiểu dữ liệu hữu ích và có ý nghĩa từ một hình ảnh hoặc một chuỗi hình ảnh. Điều này bao gồm việc thiết lập một cơ sở lý thuyết và thuật toán cho sự hiểu biết thị giác tự động.

Trong nông nghiệp, hệ thống thị giác máy tính rõ ràng là khả thi cho việc phân loại và đánh giá các đặc tính như màu sắc, hình dạng, độ dày, khuyết tật bề mặt và sự nhiễm bẩn của các sản phẩm thực phẩm thành các cấp độ khác nhau.

Phân cấp thị giác máy tính: Thị giác máy tính được phân thành ba loại cơ bản:

- Cấp thấp: Chứa hình ảnh xử lý để trích xuất đặc trưng
- Cấp trung gian: Liên quan đến nhận dạng đối tượng và cảnh 3D
- Cấp cao: Cung cấp một cái nhìn tổng quan về khái niệm của một kịch bản sự cố, mục tiêu và hành vi

Nhiệm vụ của Thị giác máy tính

- **Nhận dạng (Recognition):** Vấn đề bình thường trong thị giác máy tính, xử lý ảnh và thị giác máy (machine vision) là liệu dữ liệu hình ảnh có chứa bất kỳ đối tượng, chức năng hoặc hoạt động cụ thể nào hay không. Thông thường, một người có thể vượt qua tình huống khó xử này một cách hiệu quả và dễ dàng, tuy nhiên trong trường hợp tổng quát, thị giác máy vẫn chưa giải quyết thỏa đáng vấn đề này: các đối tượng tùy ý trong các hoàn cảnh tùy ý. Các phương pháp có sẵn để xử lý trong lĩnh vực này, tốt nhất cũng chỉ có thể giải quyết vấn đề này cho các đối tượng cụ thể, chẳng hạn như các đối tượng hình học cơ bản (ví dụ: khối đa diện), đặc điểm con người, ký tự viết tay hoặc bản thảo hoặc phương tiện giao thông, và môi trường xung quanh trong các hoàn cảnh đặc biệt thường được mô tả rõ về điều kiện ánh sáng. Có nhiều loại Nhận dạng khác nhau được chú ý như:
 - **Nhận dạng (Recognition):** Một hoặc nhiều đối tượng hoặc lớp, thường cùng với vị trí hình ảnh 2D hoặc tư thế 3D của chúng, có thể được nhận dạng trước hoặc học được trong một cảnh nhất định. Điều này có thể là bất kỳ người nào, vật phẩm nào như trái cây, rau củ, v.v.
 - **Nhận diện (Identification):** Một trường hợp riêng biệt được nhận dạng cho một đối tượng. Ví dụ: nhận diện khuôn mặt hoặc dấu vân tay của một cá nhân cụ thể hoặc nhận diện một phương tiện cụ thể.
 - **Phát hiện (Detection):** Dữ liệu hình ảnh được sàng lọc cho một tình huống cụ thể. Ví dụ: nhận diện, trong hình ảnh nông nghiệp, các tế bào hoặc mô bất thường tiềm ẩn hoặc nhận diện thu phí đường bộ tự động. Việc phát hiện bằng phép tính tương đối đơn giản và nhanh chóng thường được sử dụng để xác định các vùng nhỏ chứa dữ liệu thú vị, mà sau đó có thể được phân tích thêm để có được sự diễn giải chính xác bằng cách sử dụng các công nghệ đòi hỏi tính toán phức tạp hơn.
- **Phân tích chuyển động (Motion Analysis):** Nhiều nhiệm vụ khác nhau liên quan đến việc ước tính chuyển động khi xử lý một chuỗi hình ảnh để đo tốc độ tại mỗi điểm trong hình ảnh hoặc cảnh hình ảnh 3D, hoặc thậm chí là của máy ảnh tạo ra hình ảnh. Các ví dụ như sau:
 - **Chuyển động tự thân (Egomotion):** Xác định chuyển động cứng 3D của máy ảnh từ chuỗi hình ảnh của máy ảnh (quay và tịnh tiến).
 - **Theo dõi (Tracking):** Theo dõi (thông thường) chuyển động của một chuỗi các điểm quan tâm hoặc đối tượng trong chuỗi hình ảnh (ví dụ: ô tô, con người hoặc các sinh vật khác).
 - **Dòng quang (Optical flow):** để đánh giá cách điểm di chuyển trong mối quan hệ với mặt phẳng hình ảnh cho mỗi điểm trong ảnh, tức là chuyển động biểu kiến của nó. Chuyển động này đại diện cho cả cách điểm 3D di chuyển trong cảnh và cách máy ảnh di chuyển so với đối tượng.
- **Tái tạo cảnh (Scene Reconstruction):** Nếu một tái tạo cảnh có một hoặc (thường là) nhiều hình ảnh của một cảnh hoặc một bức tranh, nó nhằm mục đích tính toán

một mô hình 3D của cảnh đó. Mô hình có thể là một chuỗi các điểm 3D trong trường hợp đơn giản nhất. Các phương pháp chi tiết hơn tạo ra một mô hình bề mặt 3D hoàn chỉnh. Sự ra đời của hình ảnh 3D không cần chuyển động hoặc quét và các thuật toán xử lý liên quan cho phép tiến bộ nhanh chóng trong lĩnh vực này. Cảm biến lưới 3D có thể được sử dụng trong xử lý đa góc của hình ảnh ba chiều. Một số hình ảnh 3D hiện có sẵn để được hợp nhất thành các đám mây điểm và mô hình 3D.

- **Khôi phục ảnh (Image Restoration):** Mục tiêu của khôi phục ảnh là loại bỏ nhiễu khỏi ảnh (nhiều cảm biến, mờ do chuyển động, v.v.). Các loại bộ lọc đa dạng như bộ lọc thông thấp hoặc bộ lọc trung bình là giải pháp khả thi dễ dàng nhất để giảm nhiễu. Các phương pháp tiên tiến hơn được thiết kế để phân biệt giữa nhiễu và các cấu trúc hình ảnh cục bộ. Khi dữ liệu hình ảnh trước tiên được phân tích bởi các cấu trúc hình ảnh cục bộ, chẳng hạn như đường thẳng hoặc cạnh, và sau đó việc lọc được quản lý dựa trên thông tin cục bộ từ giai đoạn phân tích, mức độ loại bỏ nhiễu tốt hơn thường đạt được so với các phương pháp đơn giản hơn.

Các thách thức liên quan đến việc nhận dạng/phát hiện hình ảnh:

- Thay đổi góc nhìn
- Thay đổi tỷ lệ
- Biến thiên trong cùng một lớp
- Biến dạng hình ảnh
- Che khuất hình ảnh
- Ánh sáng
- Nền lộn xộn

Hệ thống thị giác máy tính

Cấu trúc hệ thống thị giác máy tính là rất quan trọng. Trong một số trường hợp, thiết bị là một hệ thống con của một thiết kế lớn hơn, ví dụ như với các hệ thống con để điều khiển cơ cấu truyền động cơ học, lập kế hoạch, kho lưu trữ thông tin, giao diện người-máy, v.v. Một số hệ thống có các triển khai riêng biệt, giải quyết một vấn đề đo lường và phát hiện cụ thể. Điều này cũng sẽ phụ thuộc vào việc chức năng của nó được chỉ định trước hay liệu một phần của chương trình thị giác máy tính có thể được cập nhật hoặc sửa đổi trong quá trình hoạt động hay không.

Tuy nhiên, nhiều hệ thống thị giác máy tính có các đặc điểm truyền thống.

- **Thu thập hình ảnh (Image Acquisition):** Chụp một hoặc nhiều hình ảnh bằng cách sử dụng các cảm biến hình ảnh kỹ thuật số được phát triển để bao gồm các cảm biến phạm vi, thiết bị chụp cắt lớp, radar, camera siêu âm, v.v., ngoài loại camera nhạy sáng cụ thể. Dữ liệu hình ảnh thu được phụ thuộc vào dạng cảm biến: một hình ảnh 2D cụ thể, một khối 3D hoặc một chuỗi hình ảnh. Các giá trị pixel thường đề cập đến cường độ ánh sáng trong một hoặc nhiều độ dài phổ (ánh sáng

hoặc ảnh màu), nhưng cũng có thể áp dụng cho các phép đo vật lý khác nhau, chẳng hạn như độ rộng, độ hấp thụ hoặc độ phản xạ của sóng âm hoặc sóng điện từ, hoặc cộng hưởng từ hạt nhân.

- **Tiền xử lý (Pre-Processing):** Đề xuất các mẫu thông tin nhất định, một phương pháp thị giác máy tính thường có thể được sử dụng để phân tích dữ liệu nhằm đảm bảo rằng tất cả các mong đợi mà kỹ thuật đề xuất đều có thể được đáp ứng.
- **Trích xuất đặc trưng (Feature Extraction):** Dữ liệu hình ảnh trích xuất đặc trưng ở các mức độ phức tạp khác nhau. Các ví dụ điển hình là: đường thẳng, cạnh và góc. Các điểm quan tâm cục bộ như cạnh, đốm hoặc đường thẳng. Kết cấu, hình dạng hoặc chuyển động có thể phức tạp hơn.
- **Phát hiện/Phân đoạn (Detection/Segmentation):** Có một quyết định ở một số giai đoạn trong quá trình xử lý về việc điểm hình ảnh hoặc vùng hình ảnh nào là quan trọng để xử lý thêm. Điều này có thể giống như:
 - Thứ tự của một tập hợp các điểm quan tâm cụ thể.
 - Phân đoạn một hoặc nhiều phần của hình ảnh có chứa một điểm quan tâm cụ thể.
 - Chia hình ảnh thành Kiến trúc cảnh lồng nhau bao gồm nền trước, các lớp đối tượng, các đối tượng riêng lẻ hoặc các mẫu đối tượng nổi bật (còn được gọi là hệ thống phân cấp của cảnh phân loại không gian).
 - Phân đoạn hoặc đồng phân đoạn một hoặc nhiều video thành một tập hợp các mặt nạ tiền cảnh trên mỗi khung hình trong khi vẫn duy trì tính liên tục về ngữ nghĩa thời gian của nó.
- **Xử lý cấp cao (High Level Processing):** Đầu vào thường bao gồm một tập dữ liệu nhỏ, chẳng hạn như một tập hợp các điểm hoặc một vùng hình ảnh được cho là chứa một đối tượng cụ thể. Nó giải quyết các vấn đề:
 - Kiểm tra xem dữ liệu có đáp ứng các giả định rõ ràng dựa trên mô hình và triển khai hay không.
 - Ước tính các thông số cụ thể của ứng dụng như vị trí đối tượng hoặc kích thước đối tượng.
 - Nhận dạng hình ảnh - nhóm một đối tượng được phát hiện vào các danh mục khác nhau.
 - Đăng ký hình ảnh - so sánh và tích hợp hai chế độ xem khác nhau của cùng một đối tượng.
- **Ra quyết định (Decision Making):** Đưa ra phán đoán cuối cùng cho ứng dụng,
 - Ví dụ: Ứng dụng kiểm tra tự động đạt/không đạt.
 - Khớp/không khớp trong các ứng dụng nhận dạng.
 - Gắn cờ để con người xem xét thêm trong các ứng dụng y tế, quân sự, bảo vệ và nhận dạng.

Để triển khai Thị giác máy tính, chúng ta sử dụng framework như OpenCV. Chúng ta sẽ nghiên cứu về nó trong phần tiếp theo.

OPENCV

OpenCV là một gói hoặc thư viện mã nguồn mở được phát triển bởi Intel. Thư viện này là đa nền tảng, nghĩa là chúng ta có thể sử dụng nó với java, C++, python, v.v. Trong tất cả các loại xử lý và phân tích hình ảnh và video, OpenCV hoặc phần mềm Open Source Control Vision (<http://opencv.org>) đều được sử dụng. Nó có thể xử lý hình ảnh và video để nhận dạng người, khuôn mặt hoặc thậm chí là chữ viết tay. Được tích hợp vào nhiều thư viện, bao gồm NumPy, Python có khả năng phân tích cấu trúc mảng của OpenCV. OpenCV chuyên đổi kiến thức thị giác máy sang không gian của đối tượng. Mô hình hình ảnh và chức năng từ không gian vector có thể được xác định và các phép toán toán học được thực hiện trên các đặc trưng này.

Cài đặt OpenCV

- **Trên Windows:** Dưới đây là danh sách các bước cần thiết để hoàn tất cài đặt.
 - **Cài đặt OpenCV:** Chạy lệnh “pip install opencv_python” trên Command prompt của Windows và OpenCV sẽ tự động được cài đặt.
- **Trên linux/ubuntu:**

Nhập OpenCV

Vào IDLE (môi trường lập trình Python) và nhập OpenCV:

```
>>> import cv2
```

Bạn cũng có thể kiểm tra phiên bản mình đang dùng:

```
>>> cv2.version '3.4.1'
```

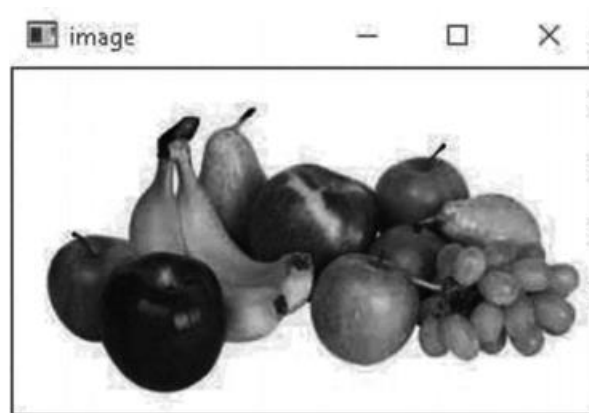
Lưu ý: Phiên bản có thể khác, tùy thuộc vào bản phát hành của nó.

Hãy thử làm gì đó với OpenCV:

Ví dụ, lưu tệp với tên opencv.py:

```
import cv2 #Thư viện OPENCV
# Tải một ảnh màu
img = cv2.imread('download.jpg')
#Ảnh phải ở cùng thư mục nơi tệp chương trình được lưu
cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Đầu ra: (Hiển thị một cửa sổ có tên 'image' chứa ảnh 'download.jpg')



(Hình 5.1. Hình ảnh đầu ra)

Chúng ta có thể chuyển đổi hình ảnh Hình 5.1 ở trên thành thang độ xám bằng cách truyền giá trị 0 vào đối số imread:

```
import cv2 #Thư viện OPENCV
# Tải một ảnh màu
img = cv2.imread('download.jpg',0)
#truyền 0 để chuyển nó thành thang độ xám
#Ảnh phải ở cùng thư mục nơi tệp chương trình được lưu
cv2.imshow('image', img)
cv2.waitKey(0)
cv2.destroyAllWindows ()
```

Đầu ra: (Hiển thị ảnh tương tự nhưng ở thang độ xám)



(Hình 5.2 Ảnh xám)

Giải thích:

- **Cv2.imread():** được sử dụng để đọc một hình ảnh. Và như đã thấy bằng cách truyền đối số thứ hai là 0, chúng ta đã chuyển đổi hình ảnh sang thang độ xám (Hình 5.2). Đối số thứ hai luôn mô tả cách hình ảnh nên được đọc:
 - **cv2.IMREAD_COLOR:** Tải một hình ảnh màu. Mọi độ trong suốt của hình ảnh đều bị bỏ qua. Đây là giá trị mặc định.
 - **cv2.IMREAD_GRAYSCALE:** Tải hình ảnh ở thang độ xám.
 - **cv2.IMREAD_UNCHANGED:** Tải hình ảnh nguyên trạng bao gồm cả kênh alpha (độ trong suốt).
- **cv2.imshow():** được sử dụng để hiển thị một hình ảnh. Cửa sổ đầu ra tự động vừa với kích thước của hình ảnh. Đối số đầu tiên là một chuỗi (string) tên cửa sổ. Đối số thứ hai là hình ảnh của chúng ta. Bạn có thể tạo bao nhiêu cửa sổ tùy thích, nhưng phải có tên cửa sổ khác nhau.
- **cv2.waitKey():** Là một chức năng chờ sự kiện từ bàn phím trong OpenCV. Đối số là thời gian tính bằng mili giây. Chức năng này chờ trong khoảng thời gian đó cho bất kỳ sự kiện bàn phím nào. Nếu bạn nhấn một phím bất kỳ trong thời gian đó, chương trình sẽ tiếp tục. Nếu đối số là 0, nó sẽ chờ một phím được nhấn mãi mãi. Bạn cũng có thể thiết lập nó để phát hiện các phím bấm cụ thể (ví dụ: khi phím 'a' được nhấn, v.v.).
- **cv2.destroyAllWindows():** Phá hủy tất cả các cửa sổ đã được tạo. Nếu muốn phá hủy một cửa sổ cụ thể, chúng ta có thể sử dụng hàm `cv2.destroywindow()` và truyền tên cửa sổ làm đối số.

OpenCV được sử dụng rộng rãi trong rất nhiều lĩnh vực như một phần quan trọng của các công nghệ khác nhau và chúng ta sẽ sử dụng OpenCV trong các chương tiếp theo cho nhiều mục đích khác nhau và rất nhiều chủ đề khác sẽ được thảo luận. Bây giờ chúng ta hãy tạo một dự án thời gian thực với sự trợ giúp của OpenCV.

Phân loại cây trồng

Trong nông nghiệp, phân loại chất lượng cây trồng là một việc rất quan trọng, và nó có thể thực hiện đơn giản bằng cách sử dụng OpenCV.

(Phần sau đây là mã Python để phân loại hạt gạo)

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

def get_classification (ratio):
    ratio = round (ratio,1)
    toret=""
    if (ratio>=3):
```



```

    toret="Slender" # (Thon)
elif (ratio>=2.1 and ratio<3):
    toret="Medium" # (Trung bình)
elif (ratio>=1.1 and ratio<2.1):
    toret="Bold" # (Mập)
elif (ratio<=1):
    toret="Round" # (Tròn)
toret=" (" + toret + ")"
return toret

print ("Starting")
img = cv2.imread ('rice.png',0) #Đường dẫn nơi hình ảnh được lưu trữ
#tải ở chế độ thang độ xám

#chuyển đổi thành nhị phân (đen trắng)
ret, binary = cv2.threshold (img,160,255,cv2.THRESH_BINARY)
# (ảnh, ngưỡng 160, giá trị gán 255, loại ngưỡng)

#bộ lọc trung bình (làm mờ, giảm nhiễu)
kernel = np.ones ((5,5), np.float32)/9
dst = cv2.filter2D (binary, -1, kernel) #-1: độ sâu của ảnh đích

kernel2 = cv2.getStructuringElement (cv2.MORPH_ELLIPSE, (3,3))
#phép co (erosion - làm mỏng đối tượng)
erosion = cv2.erode (dst,kernel2, iterations = 1)
#phép giãn (dilation - làm dày đối tượng)
dilation = cv2.dilate (erosion, kernel2, iterations = 1)

#phát hiện cạnh
edges = cv2.Canny (dilation,100,200)

### Phát hiện kích thước
#Tìm các đường viền
contours, hierarchy = cv2.findContours (erosion, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

print ("No. of rice grains=",len (contours)) #In ra số hạt gạo
total_ar = 0 #Tổng tỷ lệ khung hình
for cnt in contours:
    x,y,w,h = cv2.boundingRect (cnt) #Lấy khung chữ nhật bao quanh
    aspect_ratio = float (w)/h #Tính tỷ lệ rộng/cao
    if (aspect_ratio<1):
        aspect_ratio=1/aspect_ratio #Đảm bảo tỷ lệ luôn >= 1

```

```

total_ar+=aspect_ratio #Cộng vào tổng

avg_ar=total_ar/len (contours) #Tính tỷ lệ trung bình
print ("Average Aspect Ratio=", round (avg_ar, 2), get_classificaton (avg_ar)) #In kết
quả phân loại

#vẽ các hình ảnh (sử dụng matplotlib)
imgs_row = 2
imgs_col = 3
plt.subplot (imgs_row,imgs_col,1), plt.imshow (img,'gray')
plt.title("Original image") # (Ảnh gốc)
plt.subplot (imgs_row,imgs_col,2), plt.imshow (binary, 'gray')
plt.title("Binary image") # (Ảnh nhị phân)
plt.subplot (imgs_row,imgs_col,3), plt.imshow (dst,'gray')
plt.title("Filtered image") # (Ảnh đã lọc)
plt.subplot (imgs_row,imgs_col,4), plt.imshow (erosion, 'gray')
plt.title("Eroded image") # (Ảnh co lại)
plt.subplot (imgs_row,imgs_col,5), plt.imshow (dilation, 'gray')
plt.title("Dialated image") # (Ảnh giãn nở)
plt.subplot (imgs_row,imgs_col,6), plt.imshow (edges, 'gray')
plt.title("Edge detect") # (Phát hiện cạnh)

plt.show () #Hiển thị cửa sổ biểu đồ

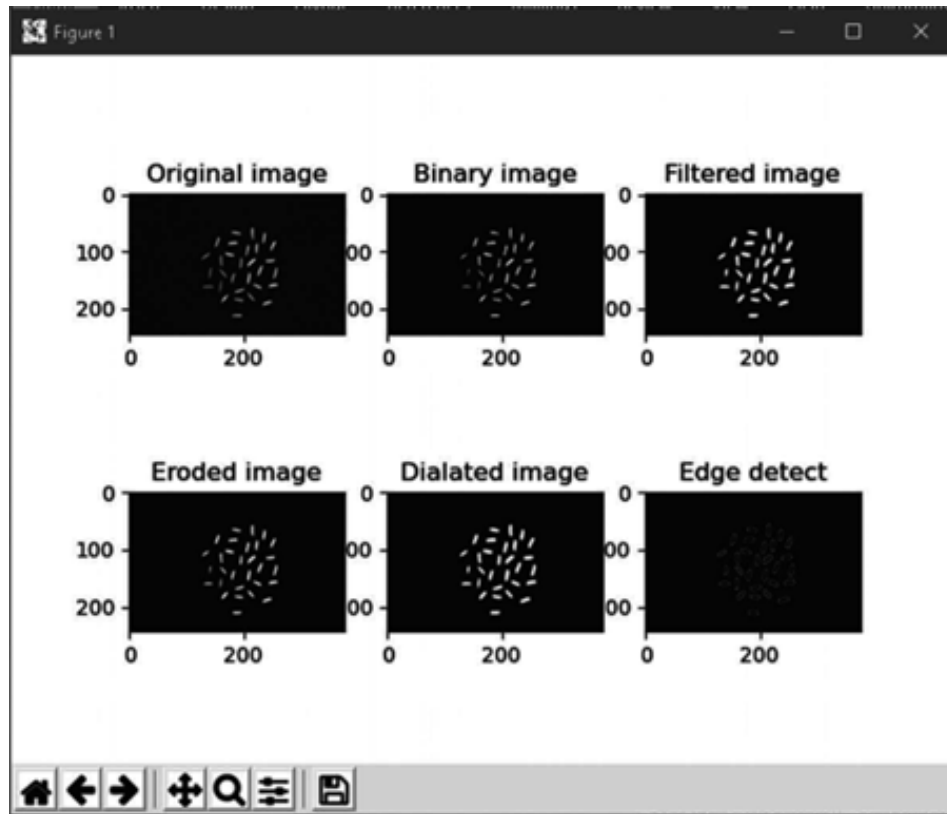
```

Đầu ra (trên cửa sổ console):

```

>>>
RESTART: E:\studies\Projects\BOOK\Rice\rice-quality-analysis-master\code.py
Starting
No. of rice grains= 30
Average Aspect Ratio= 2.3 (Medium)

```



(Hình 5.3 Đầu ra của Gạo sử dụng Matplotlib) (Hình ảnh này hiển thị 6 bước xử lý ảnh: Ảnh gốc, Ảnh nhị phân, Ảnh đã lọc, Ảnh co lại, Ảnh giãn nở, và Ảnh phát hiện cạnh)

Như chúng ta có thể thấy trong Hình 5.3, gạo đã được phát hiện và việc phân loại đã được thực hiện bằng OpenCV, được biểu diễn bằng thư viện matplotlib (thư viện này được sử dụng để trực quan hóa đầu ra sau khi phân loại gạo).

Tóm tắt

Thị giác máy tính được coi là **đôi mắt của trí tuệ nhân tạo**, có thể giúp xử lý bất kỳ video hoặc hình ảnh nào và hiểu chúng để tạo ra dữ liệu chính xác. Trong chương này, chúng ta đã thấy tầm quan trọng của thị giác máy tính và cách nó có thể hữu ích trong nông nghiệp để nhận dạng, và với hai ví dụ nhỏ (đọc ảnh và phân loại gạo), chúng ta cũng đã minh họa việc sử dụng thị giác máy tính.